



AI-Based Programming

Lab 3 : Ensemble Modeling (Voting & Stacking)

Eng. Alshaymaa Abdelhady

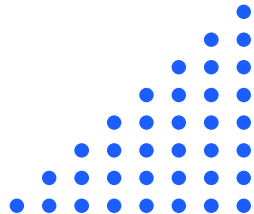
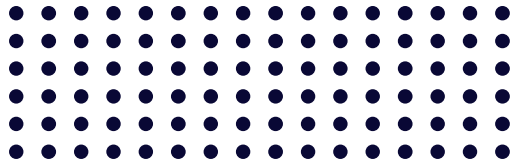




Table of contents



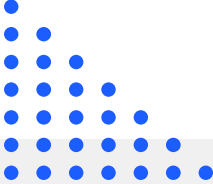
01 Introduction to
Ensemble Modeling

02 Voting Ensemble

03 Stacking

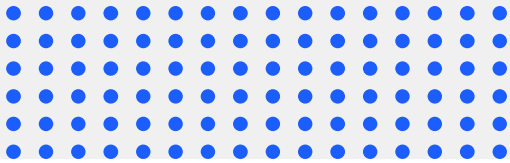
04 Base Learners

05 Modeling Workflow





01 Introduction to Ensemble Modeling



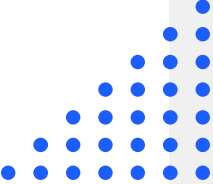


Why a Single Model is Often Not Enough

- Limitations of a Single Learner :

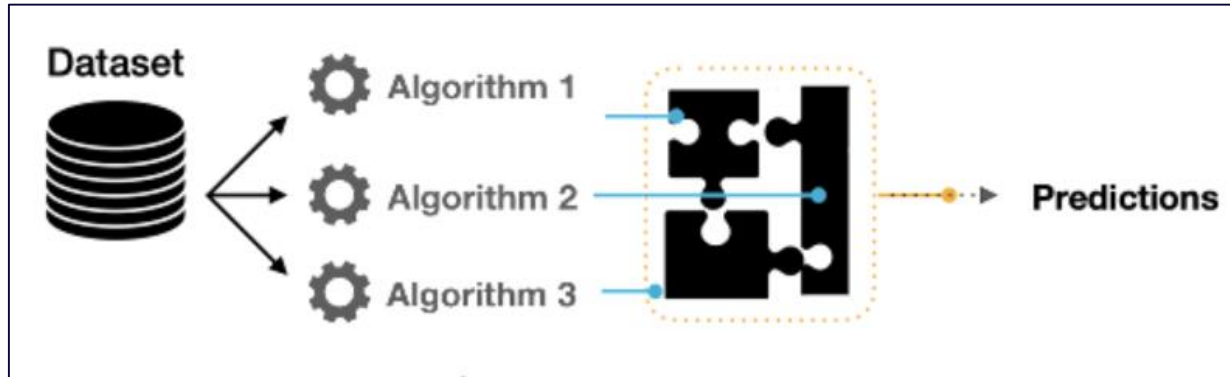
- Each algorithm captures only certain patterns (inductive bias)
- A single model can be sensitive to noise and dataset variations
- One model may perform well on some samples but poorly on others
- Relying on one learner risks unstable generalization

- Example :

- Tweet such as: “**I love the camera but hate the battery.**”
 - Contains mixed signals that remain even after feature selection and require robust modeling.
 - Single model may **focus** strongly on the word “**love**” and **ignore** the negative phrase “**hate the battery**”, leading to **misclassification**.
 - Different models may interpret such signals differently.
- 
- 

What is Ensemble Learning ?

- An ensemble model combines two or more individual machine learning models to produce a stronger predictive system.
- The individual models are called **Base Learners (or Base Estimators)**.
- Instead of relying on a single model, we aggregate predictions from multiple learners trained on the same task.

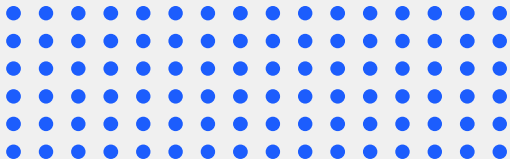


Why Ensemble Learning improves performance ?

1. Different models capture different patterns in the data
 2. Aggregation reduces individual model errors
 3. Improves predictive stability and generalization
 4. Acts as a higher-level strategy (a meta-algorithm) that coordinates multiple models
- In noisy domains like social media text, this helps balance keyword-driven predictions and contextual understanding.

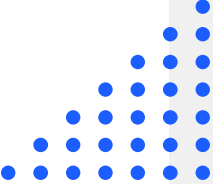


02 Voting Ensemble





Voting Ensemble

- Voting is an ensemble technique that combines predictions from multiple models to produce a final decision — similar to the “**wisdom of the crowd.**”
 - Each base learner is trained independently on the same dataset, and their outputs are aggregated to improve robustness and reduce overfitting.
 - **Why Voting Works :**
 - Different models make different errors
 - Aggregating predictions reduces individual mistakes
 - Strong models compensate for weaker ones
 - Improves prediction stability
- 
- 

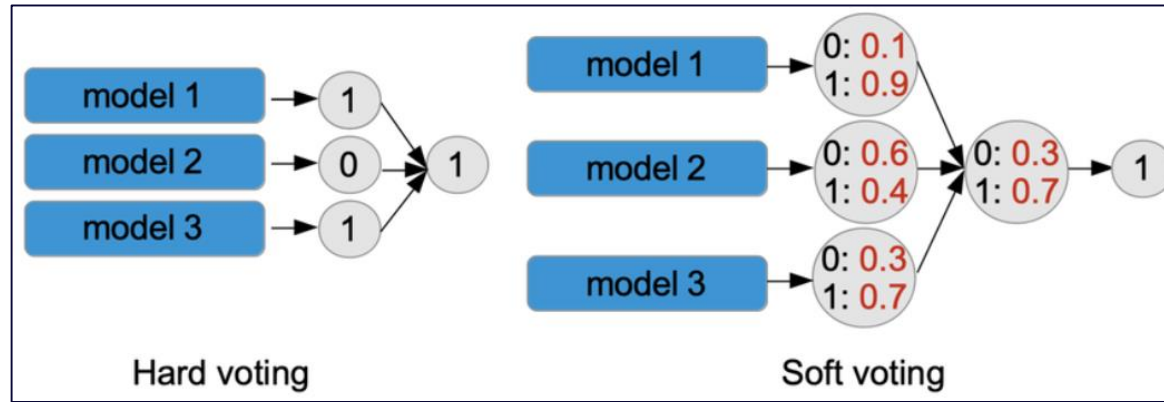
Types of Voting

1. Hard Voting (Majority Voting)

Each model predicts a class label $\rightarrow$ final class is the majority.

2. Soft Voting (Probability Averaging)

Each model predicts class probabilities $\rightarrow$ probabilities are averaged.



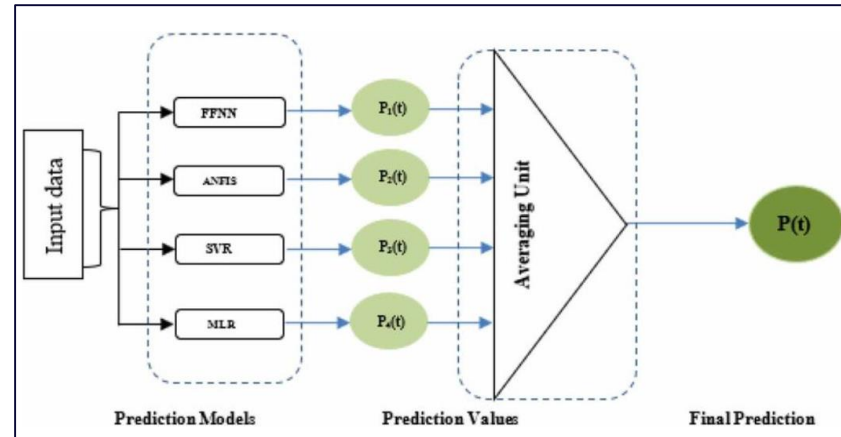
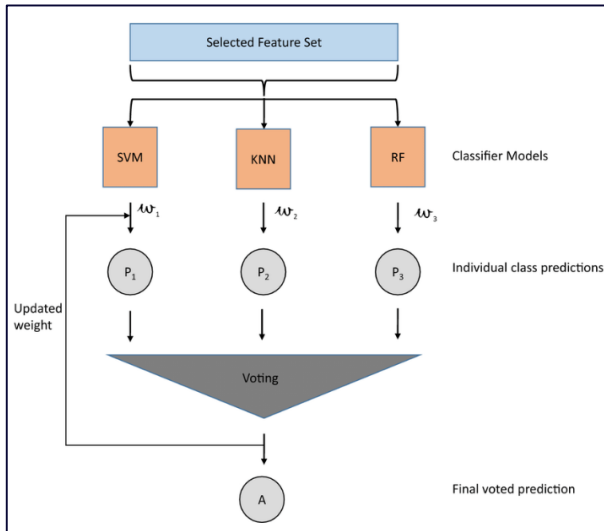
Types of Voting (Cont.)

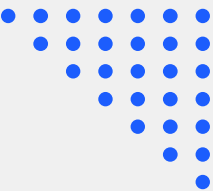
3. Weighted Voting

Models are assigned different importance based on performance.

4. Regression Voting

For regression tasks, predictions are averaged instead of voted.





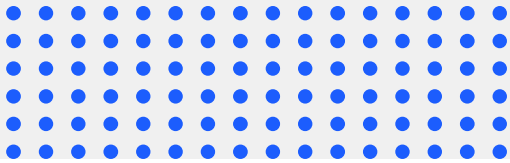
When to Use Voting ?

- Voting is effective when:
 1. You have diverse models (e.g., LR, SVM, Tree-based)
 2. Models capture different structures in the data
 3. You want a simple, stable improvement over single models
 4. Relying on one learner risks unstable generalization
- Voting assumes models are **independent contributors**.
- It does not learn how to combine them — it simply aggregates.
- This is what distinguishes Voting from Stacking



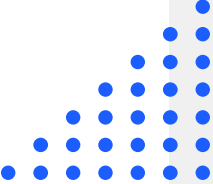


03 Stacking



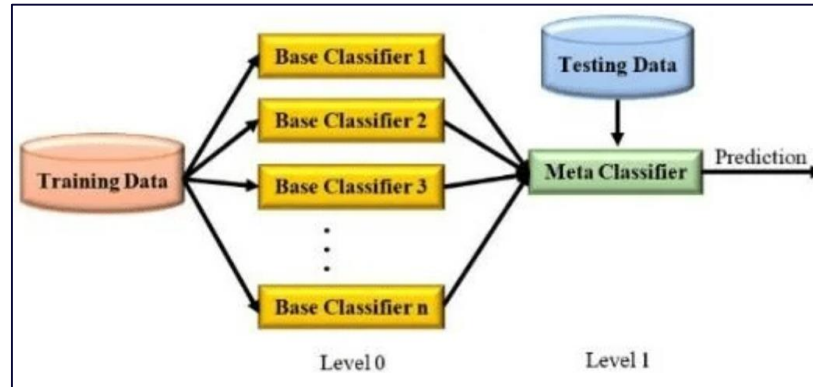


Stacking (Stacked Generalization)

- Voting Stacking is an ensemble technique where predictions from multiple base models are combined using a new model called a **Meta-Learner**.
 - Instead of applying a fixed rule (like voting), stacking **learns how to combine models optimally**.
 - **Example :**
 - One model may react strongly to emotional keywords.
 - Another may capture broader feature interactions or structure.
 - Voting merges their decisions.
 - Stacking learns when each model should be trusted more
- 
- 

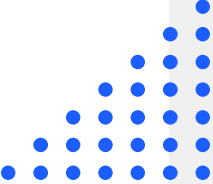
Architecture of Stacking

- **Level 0 — Base Models**
 - Multiple diverse models are trained in parallel on the same dataset.
 - Each model captures different patterns and produces predictions.
- **Level 1 — Meta-Model**
 - A second model is trained using the predictions of base models as input features (these are called **meta-features**).
 - It learns how to weight and combine base model outputs.



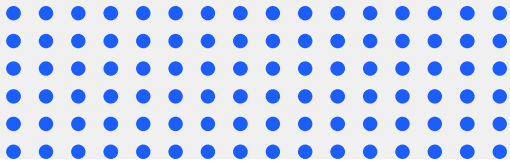


Why Cross-Validation Is Used ?

- To avoid overfitting, base models generate predictions on unseen folds (out-of-fold predictions), which are used to train the meta-model.
 - This ensures the meta-model learns generalizable relationships.
 - **Advantages :**
 - Often achieves higher accuracy than voting
 - Exploits diversity between models effectively
 - Learns context-dependent model importance
 - **Limitation :**
 - More complex to implement and tune
 - Higher computational cost (train many models)
 - Harder to interpret than simple ensembles
- 
- 

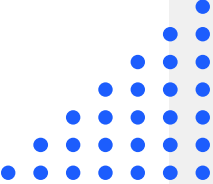


04 Base Learners





Base Learners

- For this implementation, we apply ensemble modeling using the feature representations obtained earlier (for both text and image datasets):
 - **Logistic Regression:** provides a strong linear baseline for structured feature vectors (e.g., TF-IDF for text and heuristic/GA-selected features for images).
 - **Linear SVM:** performs well in high-dimensional feature spaces and emphasizes class separation across both textual and visual descriptors.
 - **Naive Bayes:** leverages statistical evidence from feature distributions, making it effective for frequency-based text features as well as engineered image features.
 - **Why Multiple Models?**
 - Base learners form the fundamental components of an ensemble system.
 - Each model interprets the data differently due to its underlying assumptions, resulting in diverse prediction behaviors.
 - This diversity allows the ensemble to reduce individual model errors and produce a more robust final prediction.
- 
- 

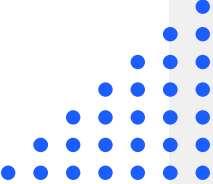


Base Learners (Cont.)

- **Heterogeneous Ensemble**

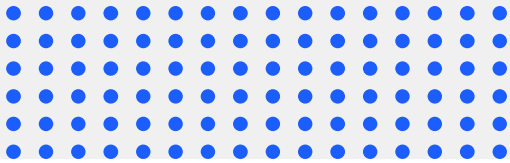
- We combine different algorithms rather than repeating the same one.
- This is known as a heterogeneous ensemble, where diversity is introduced through distinct learning mechanisms and inductive biases.

- **Example:**

- **Naive Bayes** relies on statistical feature distributions (e.g., word frequencies in text or intensity/descriptor frequencies in images), while **SVM** captures separation patterns across many features simultaneously, and **Logistic Regression** provides globally balanced feature weighting.
 - These differences create the complementary behavior required for effective ensemble learning across both textual and visual datasets.
- 
- 



05 Modeling Workflow





Modeling Workflow

- **Implementation Steps:**

- Capture selected features (GA output)
- Train/validation split
- Define evaluation function
- Train base learners (baseline)
- Voting ensemble
- Stacking ensemble
- Compare results

- We demonstrate this workflow using the Sentiment140 dataset as a real-world case study of unstructured text classification.

- 1.6 million real tweets
 - Binary sentiment labels (positive / negative)
 - High linguistic variability
 - Informal structure and noisy expressions
 - No single model can reliably interpret all linguistic patterns, making it ideal for studying model complementarity.
- 
- 

Ensemble Modeling Implementation(Text dataset)

1. Capture selected features (GA output)

- We start from the optimized feature matrix obtained through GA-based feature selection.
- No additional preprocessing is performed here — the goal is to focus purely on modeling.
- We rename variables for clarity and verify the feature space before training.

```
# Assumed output from Lab2:  
# X_selected, mask, score = feature_selection_pipeline(X_text, y, method="ga")  
  
# Rename to keep Text application explicit and avoid confusion later  
X_text_sel = X_selected  
y_text = y  
  
# Sanity checks  
print("Text selected feature space shape:", X_text_sel.shape)  
print("Feature selection score from Lab2:", score)  
print("Selected features count:", (mask == 1).sum())  
  
# Optional: confirm label set (useful if students try multi-class datasets)  
print("Unique labels:", sorted(set(y_text)))
```

Ensemble Modeling Implementation(Text dataset) (Cont.)

2. Preparing Training and Validation Sets

- The selected features are split into training and validation sets.
- All models will be evaluated on the same validation data to ensure a fair comparison.
- Stratified splitting preserves the class distribution across both sets.

```
from sklearn.model_selection import train_test_split

X_train_t, X_val_t, y_train_t, y_val_t = train_test_split(
    X_text_sel, y_text,
    test_size=0.2,
    random_state=42,
    stratify=y_text  # keeps class distribution consistent
)

print("Train shape (Text):", X_train_t.shape)
print("Val shape (Text): ", X_val_t.shape)
```

Ensemble Modeling Implementation(Text dataset) (Cont.)

3. Unified Evaluation for Binary and Multi-Class Problems

- We define a single evaluation function used across all models. Accuracy works for any classification task, while F1-score requires specifying an averaging strategy when multiple classes exist.
- For binary classification → standard F1 is used.
- For multi-class classification → weighted F1 is applied to account for class imbalance.
- This makes the evaluation function reusable across different datasets

```
from sklearn.metrics import accuracy_score, f1_score
import numpy as np

def evaluate_classifier(model, X, y, average_multi="weighted"):
    """
    Evaluate a trained classifier on a dataset.

    Returns:
    - accuracy: overall correctness (works for binary & multi-class)
    - f1-score:
        * If binary (2 classes): standard F1 is used.
        * If multi-class: an averaging strategy is required.

    average_multi options (for multi-class):
    - "macro" : treats all classes equally (good for minority-class fairness)
    - "weighted" : weighted by class support (good when classes are imbalanced)
    - "micro" : global counting (often close to accuracy in some settings)
    """
```

```
y_pred = model.predict(X)
acc = accuracy_score(y, y_pred)

n_classes = len(np.unique(y))
if n_classes == 2:
    f1 = f1_score(y, y_pred) # binary default
else:
    f1 = f1_score(y, y_pred, average=average_multi)

return acc, f1
```

Ensemble Modeling Implementation(Text dataset) (Cont.)

4. Training Base Learners (Baseline Models)

- We train three complementary algorithms on the same feature space:
 1. Logistic Regression captures global linear relationships.
 2. Linear SVM focuses on margin-based separation in high-dimensional spaces.
 3. Naive Bayes relies on statistical word-frequency evidence.
- These models create diverse prediction behaviors, forming the foundation for ensemble learning

```
from sklearn.linear_model import LogisticRegression
from sklearn.svm import LinearSVC
from sklearn.naive_bayes import MultinomialNB

# Base learners chosen for sparse text features (TF-IDF-like)
lr_t = LogisticRegression(max_iter=2000) # strong linear baseline
svm_t = LinearSVC() # often strong for high-dimensional sparse text
nb_t = MultinomialNB() # effective for frequency-based evidence

# Train
lr_t.fit(X_train_t, y_train_t)
svm_t.fit(X_train_t, y_train_t)
nb_t.fit(X_train_t, y_train_t)
```

```
# Evaluate (baseline) - works for binary & multi-class
acc_lr, f1_lr = evaluate_classifier(lr_t, X_val_t, y_val_t)
acc_svm, f1_svm = evaluate_classifier(svm_t, X_val_t, y_val_t)
acc_nb, f1_nb = evaluate_classifier(nb_t, X_val_t, y_val_t)

print("=== Baseline (Single Models) ===")
print(f"Logistic Regression: acc={acc_lr:.4f}, f1={f1_lr:.4f}")
print(f"Linear SVM : acc={acc_svm:.4f}, f1={f1_svm:.4f}")
print(f"Naive Bayes : acc={acc_nb:.4f}, f1={f1_nb:.4f}")
```


Ensemble Modeling Implementation(Text dataset) (Cont.)

5. Building the Voting Ensemble

- We implement Hard Voting, where each model predicts a class label and the final decision is determined by majority agreement.
- This approach improves robustness because errors from one model can be compensated by others.

```
from sklearn.ensemble import VotingClassifier

# Hard Voting: majority vote over predicted labels
# Note: We rebuild fresh estimators here (clean ensemble training)
voting_t = VotingClassifier(
    estimators=[
        ("lr", LogisticRegression(max_iter=2000)),
        ("svm", LinearSVC()),
        ("nb", MultinomialNB())
    ],
    voting="hard"
)

# Train ensemble
voting_t.fit(X_train_t, y_train_t)

# Evaluate - works for binary & multi-class
acc_v, f1_v = evaluate_classifier(voting_t, X_val_t, y_val_t)

print("=== Voting Ensemble ===")
print(f"Voting (Hard): acc={acc_v:.4f}, f1={f1_v:.4f}")
```

Ensemble Modeling Implementation(Text dataset) (Cont.)

6. Building the Stacking Ensemble

- Stacking introduces a second-level model (meta-learner).
Instead of using a fixed rule, the meta-model learns how to combine predictions from base learners.
- Internal cross-validation is used to generate reliable meta-features and reduce overfitting.

```
from sklearn.ensemble import StackingClassifier

stack_t = StackingClassifier(
    estimators=[
        ("lr", LogisticRegression(max_iter=2000)),
        ("svm", LinearSVC()),
        ("nb", MultinomialNB())
    ],
    final_estimator=LogisticRegression(max_iter=2000), # meta-learner
    cv=5 # internal CV to generate out-of-fold meta-features and reduce overfitting
)

# Train stacking model
stack_t.fit(X_train_t, y_train_t)

# Evaluate - works for binary & multi-class
acc_s, f1_s = evaluate_classifier(stack_t, X_val_t, y_val_t)

print("=== Stacking Ensemble ===")
print(f"Stacking: acc={acc_s:.4f}, f1={f1_s:.4f}")
```

Ensemble Modeling Implementation(Text dataset) (Cont.)

7. Comparing Single Models vs Ensemble Models

- We summarize all results in one table to compare:
 1. Individual learners (baseline)
 2. Voting ensemble
 3. Stacking ensemble
- This comparison demonstrates whether combining models leads to measurable performance gains

```
import pandas as pd

text_results = pd.DataFrame([
    ["Logistic Regression", acc_lr, f1_lr],
    ["Linear SVM", acc_svm, f1_svm],
    ["Naive Bayes", acc_nb, f1_nb],
    ["Voting (Hard)", acc_v, f1_v],
    ["Stacking", acc_s, f1_s],
], columns=["Model", "Accuracy", "F1-score"])

print(text_results)
```

	Model	Accuracy	F1-score
0	Logistic Regression	0.8732	0.8710
1	Linear SVM	0.8895	0.8881
2	Naive Bayes	0.8420	0.8392
3	Voting (Hard)	0.8950	0.8934
4	Stacking	0.9023	0.9010